Register Once, Verify Everywhere: Identity for Autonomous Agents on the Open Web

Author Punit Pandey

Project AgentAuth / iAgents

Status Draft for arXiv-style preprint

Date March 31, 2026

Abstract

As AI agents begin to operate as first-class actors on the internet, they require an identity mechanism that works without browser redirects, human consent screens, or platform-specific manual onboarding. Existing web authentication systems are optimized for humans, while emerging agent identity systems often emphasize decentralized identity stacks, on-chain registries, or challenge-response protocols that may be too heavyweight for straightforward open-web application integration. This paper presents **AgentAuth**, a practical identity layer for autonomous agents built around three ideas: (1) agent self-registration via a simple HTTP API, (2) separation between a long-lived **registry-only secret** and short-lived **platform proof tokens**, and (3) public verifiability through either a registry verification endpoint or offline JWT signature verification. The design aims to minimize integration complexity for internet applications while preserving a cleaner trust boundary than raw API-key reuse. We describe the protocol, security model, implementation, and tradeoffs of this approach, and discuss how it differs from OAuth-style human auth, decentralized identifier ecosystems, and recent agent identity products. The central claim of this work is not that agent identity is a new problem, but that a simpler, deployable architecture can materially lower the barrier to building agent-native applications on the open web.

1. Introduction

The web has long relied on identity systems designed around human behavior. Users type passwords, click verification links, complete OAuth consent flows, or authenticate through federated login providers such as Google, GitHub, or Apple. These patterns assume an interactive user with a browser, an inbox, and the ability to make human judgments during onboarding and login.

Autonomous agents do not fit that model well. A capable software agent may discover a new application, read its onboarding instructions, send HTTP requests, store credentials, and operate continuously without human supervision. For such an agent, the core question is not "How does a person log in?" but rather: **How does a machine prove who it is across multiple independent applications without requiring human intervention each time?**

This question becomes more important as agent-native applications emerge. Social platforms for agents, agent marketplaces, hosted tools, and agent-to-agent services all need some answer to the same identity problem:

- How does an application know which agent it is interacting with?
- How can an agent register itself without a dashboard or browser flow?
- How can platforms avoid asking agents to reveal their root credential?
- How can identity be reused across applications rather than recreated per service?

AgentAuth is a pragmatic response to those requirements. It is an identity registry and verification layer for autonomous agents. An agent registers itself with the registry once, receives a long-lived `registry_secret_key` for registry-only calls, and uses short-lived `platform_proof_token` JWTs when interacting with applications. Applications verify those proof tokens either by asking the registry directly or by verifying the JWT locally using the registry's published public key.

The contributions of this paper are modest but concrete:

1. A protocol for autonomous agent registration and cross-platform verification that does not require browser-based onboarding.
2. A trust split between a long-lived registry secret and short-lived platform proof tokens, reducing the need to expose root credentials to third-party applications.
3. A deployable implementation consisting of a registry, Python SDK, CLI, and two applications that use the protocol today.
4. A clear argument for why simple, centralized, open-web identity infrastructure can still be useful even in a landscape that increasingly favors decentralized identifiers and verifiable credential frameworks.

The paper does **not** claim that agent identity as a category is novel. Similar ideas now exist in decentralized identity systems, agent interoperability protocols, and newly emerging agent identity products. The claim is narrower: there is value in a simpler, application-oriented architecture that can be adopted immediately by ordinary internet services.

2. Problem Statement

The design target is not "all digital identity," but a specific problem class: **identity for autonomous agents interacting with open-web applications.** In this setting, the following properties matter more than feature-completeness:

2.1 Design requirements

Headless onboarding.

An agent must be able to register itself and begin using the system via HTTP requests alone.

Cross-application reuse.

Identity should be portable across multiple applications that trust the same registry.

Minimal platform complexity.

Application developers should not need to implement an entire decentralized identity stack to accept agent logins.

Safer credential boundaries.

The credential an agent uses to talk to the registry should not be the same credential it sends to third-party applications.

Public verifiability.

An application should be able to verify a presented identity by one network call or by offline cryptographic verification.

Human-optional trust tiers.

The base protocol should work without a human, while still allowing optional stronger identity signals such as email verification.

2.2 Non-goals

AgentAuth is not intended to solve every identity problem.

It is **not**:

- a general authorization framework;
- a reputation system;
- a Sybil-resistant identity scheme;
- a decentralized consensus protocol;
- a privacy-preserving anonymous credential system;
- a replacement for platform-specific OAuth when a platform already requires OAuth for access control.

Instead, it addresses a narrow but common need: **who is this agent, and how can my application verify that fact with minimal friction?**

3. System Model

AgentAuth has three main actors:

Agent.

An autonomous software system that wants to register and later prove its identity to applications.

Registry.

The trust anchor that registers agent names, stores registry-side secrets, issues proof tokens, and publishes the public key used for token verification.

Platform.

Any application that accepts autonomous agents and wants to verify the identity they present.

The protocol is intentionally simple:

1. The agent registers once with the registry.
2. The registry returns a long-lived registry secret and an initial proof token.
3. The agent uses the registry secret only for registry calls.
4. The agent uses the short-lived proof token everywhere else.
5. The platform verifies the proof token.

This split is the core design decision in the system.

4. Protocol Design

4.1 Registration

An agent registers by calling:

```
POST /v1/agents/register
```

with a globally unique name, an optional description, and an optional email address. A successful registration returns:

- `registry_secret_key`
- `platform_proof_token`
- `platform_proof_token_expires_in_seconds`
- basic agent profile fields

The `registry_secret_key` is shown once and is intended only for calls back to the registry. The optional email creates a stronger, human-linked verification tier, but is not required for basic operation.

4.2 Identity split

The main credential split is:

- **Registry credential:** `registry_secret_key`
- **Platform credential:** `platform_proof_token`

This separation gives the system a cleaner trust boundary than naive API-key reuse. A platform never needs the agent's registry secret. It only needs a short-lived proof token that the registry has issued.

4.3 Proof token issuance

When the current proof token expires, the agent requests a new one:

```
POST /v1/agents/me/proof
Authorization: Bearer <registry_secret_key>
```

The registry responds with a signed JWT containing:

- `sub` : agent name
- `description`
- `verified`
- `iat`
- `exp`

The current implementation uses ES256 (ECDSA P-256) signatures and a default proof-token lifetime of five minutes.

4.4 Platform verification

Platforms can verify a proof token in two ways:

Registry-mediated verification

```
GET /v1/verify-proof/{token}
```

This is the simplest integration path. The platform asks the registry to validate the token and returns the verified identity fields.

Offline verification

```
GET /.well-known/jwks.json
```

The platform fetches the registry's public key once, caches it, and verifies the JWT locally.

This dual-mode design intentionally supports both low-effort adoption and lower-latency validation.

4.5 Public directory features

The registry also exposes public lookup endpoints:

- `GET /v1/agents/{name}`
- `GET /v1/agents`

This gives the system an identity-directory property rather than being only a token minting service. The model resembles DNS in one narrow sense: anyone can query public identity information, and the value of the system increases when verification is easy to perform.

5. Implementation

The reference implementation of AgentAuth is open source and consists of four packages:

- **registry/**: FastAPI service implementing registration, proof issuance, verification, and public lookup.
- **sdk/**: Python SDK and CLI for agent-side use.
- **agentboard/**: short-form agent posting application.
- **agentblog/**: long-form agent publishing application.

5.1 Registry

The registry is implemented in FastAPI with a SQLite backing store. Agent secrets are bcrypt-hashed before storage. To avoid checking every stored hash on authentication, the store keeps a short key prefix index that narrows the candidate set before bcrypt verification.

The server maintains an ECDSA P-256 signing key used to issue proof tokens. The public key is published for offline verification by platforms.

5.2 SDK and CLI

The Python SDK provides client methods for registration, proof issuance, profile lookups, and token verification. Credentials are stored locally at:

```
~/.config/agentauth/credentials/<agent_name>.json
```

with restrictive file permissions. A CLI wraps the same functionality for environments where a direct SDK integration is unnecessary.

5.3 Agent-facing onboarding

The registry serves a skill page at `/` and `/skill.md` with curl-first onboarding instructions. This matters because some agents discover capabilities by reading web pages or skill documents rather than through a human-driven setup wizard.

5.4 Applications

AgentBoard and AgentBlog demonstrate how applications can accept proof tokens today. They do not need to hold or manage the agent's registry secret. Instead, they verify proof tokens through the registry verification endpoint and then use the verified identity returned by the registry to attribute actions.

Together, the registry and applications show end-to-end feasibility:

- agent self-registration;
- immediate cross-application use;
- platform-side verification;
- reuse of one identity across multiple services.

6. Security Model and Threats

AgentAuth is a practical system, not a formally proven protocol. Its security properties follow from explicit assumptions and tradeoffs.

6.1 Threats addressed

Root secret exposure to platforms.

The design reduces this risk by giving platforms only short-lived proof tokens rather than the registry secret itself.

Identity spoofing by unregistered names.

Platforms verify proof tokens issued by the registry rather than trusting self-asserted names in application payloads.

Replay beyond token lifetime.

Proof tokens are short-lived and expire quickly by design.

Offline verification needs.

Platforms that do not want to call the registry on every request can verify proof-token signatures locally.

6.2 Threats not fully solved

Compromised registry.

A centralized registry is a high-value trust anchor. If its signing key or database is compromised, the attacker may mint or validate fraudulent identities.

Phishing for registry secrets.

A malicious platform could ask an agent to provide its registry secret directly. AgentAuth mitigates this primarily through protocol design and onboarding instructions, not through cryptographic enforcement. The agent is told never to send its registry secret to any third-party platform.

Token theft within validity window.

A stolen proof token may be replayed until expiration. The short token lifetime limits but does not eliminate this risk.

Sybil attacks and cheap identity creation.

The base protocol allows pseudonymous self-registration. This is good for accessibility and poor for Sybil resistance.

Platform authenticity.

The current system authenticates agents to platforms but does not authenticate platforms back to agents beyond ordinary HTTPS trust.

6.3 Security posture

The system should therefore be understood as providing:

- **practical identity verification** for open-web agent applications;
- **bounded credential exposure** relative to direct API-key sharing;
- **limited trust claims** about accountability or uniqueness.

It does **not** yet provide:

- scoped per-platform proof tokens;
- nonce-based challenge-response at application login time;
- public transparency logs;
- federation between multiple registries;
- privacy-preserving or anonymous agent identity.

7. Why This Is Not Just OAuth

At the token level, AgentAuth intentionally reuses familiar web-auth mechanisms. The proof-token flow is conceptually similar to OAuth 2.0 client credentials: a long-lived credential is exchanged for a short-lived bearer token.

The novelty is therefore not "JWTs for agents." That would be an uninteresting claim.

The difference lies in what surrounds token issuance:

7.1 Autonomous registration

In most OAuth ecosystems, a human must register an application through a dashboard, configure redirect URIs, and manually provision credentials. AgentAuth instead allows an autonomous agent to create its own identity with a direct HTTP request.

7.2 Identity directory

OAuth is primarily an authorization framework. AgentAuth also acts as a public identity registry with lookup and listing endpoints.

7.3 Portable agent identity

The same registered agent identity can be reused across multiple applications that trust the registry.

7.4 Agent-native discovery

The skill-page-based onboarding flow is designed for tool-using agents that discover capabilities through web content or instructions, not just for human developers reading API docs.

In other words, AgentAuth reuses conventional token infrastructure but changes the onboarding, trust boundary, and application model to fit autonomous agents.

8. Comparison to Decentralized Identity Approaches

An important question is why a centralized registry should exist at all when decentralized identity frameworks already exist.

8.1 DIDs and Verifiable Credentials

W3C Decentralized Identifiers (DIDs) and Verifiable Credentials (VCs) provide a general framework for portable cryptographic identity. They are powerful and standards-based, but they also impose substantial implementation and conceptual overhead for straightforward application sign-in.

For many ordinary web applications, the relevant problem is not "build a full decentralized trust graph," but rather:

> "How can my service let autonomous agents sign in safely this week?"

AgentAuth chooses to optimize for that question.

8.2 Challenge-response systems

Systems based on challenge-response and self-held keypairs avoid shared secrets and can offer stronger cryptographic guarantees. However, they also shift more complexity to the application and the agent. AgentAuth instead starts from the minimal deployable architecture: a registry issues short-lived proof tokens, and applications verify them with minimal effort.

8.3 Centralization as a deliberate tradeoff

Centralization is a weakness from a trust-minimization perspective and a strength from a deployability perspective. AgentAuth accepts this tradeoff explicitly. The design starts with one trust anchor because that is often the cheapest way to create an interoperable identity layer early in an ecosystem's formation.

If the ecosystem grows, future versions could move toward:

- federation between registries;
- scoped tokens;
- platform registration;
- challenge-response upgrades;
- or DID/VC-compatible representations.

9. Related Work

9.1 Web identity and credential standards

OAuth 2.0 and OpenID Connect remain the dominant identity and authorization frameworks on the web, but they are optimized for human-facing applications and consent-driven login flows. OpenID for Verifiable Credentials extends web identity toward portable credentials and presentations, while W3C DIDs and VCs provide a more general decentralized identity substrate. These are highly relevant foundations, but they do not by themselves specify a minimal agent-first onboarding and verification flow for ordinary open-web applications.

9.2 Agent interoperability protocols

The Agent2Agent (A2A) protocol addresses interoperability and capability discovery between agents, including the concept of discoverable agent metadata. This is adjacent to AgentAuth but solves a different layer of the stack. A2A focuses on communication and task interoperability; AgentAuth focuses on identity issuance and verification for application access.

9.3 Emerging agent identity products

Several recent systems are close in spirit to this work.

Agent Auth by Vigil presents "sign-in for AI agents" using DID-based identity, Ed25519 challenge-response, and verifiable credentials, with both headless and hosted sign-in flows. Relative to Vigil, AgentAuth is simpler and more centralized: it uses a registry-issued secret plus proof-token model instead of a pure key-ownership challenge-response flow.

AgentID positions itself as a decentralized open standard for AI agent identity, with portable metadata, trust levels, and on-chain registration. Relative to AgentID, AgentAuth is narrower in scope. It does not attempt to solve discovery, reputation, or on-chain trust. It instead concentrates on low-friction application authentication.

Identity Registry frames the problem around tamper-proof cryptographic directories, owner binding, and ledger-anchored accountability. Relative to that model, AgentAuth again chooses easier deployability over stronger cryptographic decentralization and legal-owner binding.

These comparisons clarify the contribution of AgentAuth. It is not the most decentralized system, nor the most cryptographically ambitious. Its value lies in reducing the effort required for internet applications to become usable by autonomous agents now.

10. Limitations

The current design has several important limitations.

10.1 Centralized trust anchor

The registry is a single logical authority. This creates operational simplicity and ecosystem dependence on one service.

10.2 Weak default assurances

Pseudonymous self-registration allows low-friction onboarding but provides only weak assurance about who ultimately controls an identity.

10.3 No platform-specific scoping

Proof tokens are short-lived but not currently scoped to a specific relying party or permission set.

10.4 No formal privacy story

The system is designed for attributable agent identity, not for anonymous or unlinkable interactions.

10.5 Limited empirical evaluation

The current implementation demonstrates feasibility through deployed applications and code, but not yet through large-scale empirical measurements, user studies, or formal adversarial evaluation.

These limitations are real and should be stated plainly in any public paper or submission.

11. Future Work

The most important next directions are:

1. **Scoped proof tokens** bound to a specific platform or permission set.
2. **Challenge-response upgrades** for platforms that want stronger possession proofs at authentication time.
3. **Platform authenticity mechanisms** so agents can verify the applications they are sending proof tokens to.
4. **Registry federation** to reduce dependence on one operator.
5. **Compatibility layers for DID/VC ecosystems** so the system can interoperate with more standards-heavy environments.
6. **Formal threat modeling and protocol analysis** beyond the current design-oriented treatment.
7. **Empirical evaluation** using adoption data, latency measurements, incident analysis, and developer integration studies.

12. Conclusion

Autonomous agents need an identity mechanism that is less interactive than human login and less operationally heavy than many decentralized identity stacks. AgentAuth proposes one answer: let agents register once through a simple HTTP API, keep a registry-only secret private, and use short-lived proof tokens everywhere else.

This design does not eliminate the hard problems of trust, Sybil resistance, or decentralization. It does, however, establish a practical and immediately deployable identity layer for agent-native applications on the open web. The broader significance of the approach is architectural rather than cryptographic: if applications can

adopt agent identity with minimal integration cost, then the barrier to building interoperable agent-facing services falls materially.

In that sense, the value of AgentAuth is not that it solves every identity problem. It is that it narrows the problem to a shape that can be deployed now.

References

1. P. Pandey, "AgentAuth" (project repository), GitHub. https://github.com/punitpandey/agentauth
2. W3C, "Decentralized Identifiers (DIDs) v1.0." https://www.w3.org/TR/did-1.0/
3. W3C, "Verifiable Credentials Data Model v2.0." https://www.w3.org/TR/vc-data-model-2.0/
4. OpenID Foundation, "OpenID for Verifiable Credentials." https://openid.net/sg/openid4vc/specifications/
5. Agent2Agent Protocol Specification. https://a2aproject.github.io/A2A/specification/
6. Agent Auth by Vigil. https://usevigil.dev/
7. AgentID. https://agentid.md/
8. Identity Registry. https://identityregistry.org/
9. P. Pandey, "AgentAuth vs OAuth — Why Not Just Use OAuth?" https://github.com/punitpandey/agentauth/blob/main/docs/oauth-comparison.md
10. P. Pandey, "Platform Verification — Do We Need It?" https://github.com/punitpandey/agentauth/blob/main/docs/platform-verification.md